

esumo destilado do workflow para integrar Aider ao Emacs usando o Aidermacs:

1. Preparação do Aider (Terminal)

Instalação: Siga as instruções oficiais do site do Aider para o seu sistema (0:23).

Verificação: Confirme a instalação com `aider --version` (0:39).

Modelos e Chaves: Obtenha uma chave de API (ex: OpenRouter) e configure a variável de ambiente: `export OPENROUTER_API_KEY=...` (1:52).

Teste Inicial: Inicie um projeto e teste o comando `/ask` para garantir que o Aider responde corretamente antes de prosseguir (3:15).

2. Configuração do Emacs (Aidermacs)

Pacote: Utilize o `aidermacs` (via `straight.el` ou outro gerenciador de pacotes) (3:49).

Bindings: Configure um atalho, como `C-c c a`, para chamar `aidermacs-transient-menu` (3:52).

Autenticação: Defina a variável `OPENROUTER_API_KEY` dentro do Emacs (sugestão: use `password-store` para segurança) (4:12).

Customização: Ajuste as variáveis `aidermacs-default-chat-mode` (para 'code') e `aidermacs-default-model` (ex: DeepSeek R1) no seu arquivo de configuração (4:24).

3. Fluxo de Trabalho (Uso Diário)

Menu Principal: Use `C-c c a` para abrir o menu do Aidermacs (5:02).

Edição: Selecione `code change` (tecla `C`) para solicitar alterações no código diretamente pelo chat (5:13).

Arquitetura: Se necessário, use o `architect mode` para separar modelos de raciocínio e modelos de edição (5:57).

Iteração: O Aider cria/modifica arquivos no seu projeto; revise as sugestões e aplique as mudanças no seu código (5:24).

Emacs no terminal foca em simplicidade e integração com o sistema. Abaixo, o destilado das etapas, comandos e filosofia:

Workflow: Emacs no Terminal

Escolha do Terminal Emulador (3:13)

 iTerm2: A escolha do autor para macOS, valorizada por gerenciar abas e janelas de forma eficiente.

✨ Ghostty: Recomendado como uma alternativa moderna e performática, disponível para Mac e Linux.

Comandos Essenciais de Inicialização (3:33)

 `emacs -nw` (No Window): Comando fundamental para iniciar o Emacs dentro da interface do terminal, dispensando a GUI.

 `--init-directory`: Flag poderosa (3:59) que permite apontar para um diretório de configuração dedicado (ex: `~/.emacs.d/emacs`), facilitando a gestão de múltiplos ambientes.

Filosofia de Configuração (4:31)

 Declaração de Falência (Bankruptcy): O autor recomenda abandonar configurações inchadas (migrou de >2.000 linhas para ~150 linhas).

 Minimalismo: Remover logos, ícones e elementos gráficos que não agregam valor funcional.

 Migração seletiva: Transferir apenas o que é essencial para o fluxo de trabalho diário, ignorando o que é supérfluo.

Considerações e Trade-offs (0:36)

 Perdas: Perda de suporte a imagens (Org mode), visualização nativa de PDFs e limitações de atalhos (tecla Command capturada pelo terminal).

 Ganhos: Integração fluida com o workflow de linha de comando, estabilidade no uso de LSP (Language Server Protocol) e foco em edição de arquivos remotos.

uso de plugins de IA para Emacs para integrar o Aider (uma ferramenta de linha de comando para codificação assistida por IA) diretamente ao editor. Abaixo, o destilado do workflow e das ferramentas exploradas:

Bibliotecas e Ferramentas

Aider (Core): O motor de IA baseado em Python que realiza a comunicação com LLMs, executa comandos de shell, testes e gerencia o contexto do repositório (0:48, 1:16).

Aider.el: Plugin principal, mais robusto e com foco em metodologia, incluindo suporte a TDD (Test-Driven Development) (0:40, 5:44, 7:15).

Aider-max: Alternativa mais minimalista focada em integrar o Aider com uma interface estilo Magit, visando preencher a lacuna deixada pelo Cursor (0:59, 9:45).

Tree-sitter: Utilizado para fornecer informações sintáticas sobre a base de código para a IA (1:53).

Workflow e Comandos

Interface de Controle (Hydra/Transient): Ambos os plugins utilizam menus do tipo transient/hydra para disparar comandos rapidamente (8:43, 10:18).

Ciclo de TDD: O Aider.el guia o usuário pelo fluxo red-green-refactor (7:18).

Comandos de Contexto:

add/drop files: Gerenciamento dos arquivos enviados ao chat (10:36, 19:41).

send line/region: Envio de trechos específicos de código ou blocos para a IA (26:01).

architect/editor mode: Modo experimental que separa modelos de raciocínio de modelos de edição (12:26).

legacy code techniques: Implementação de padrões como o sprout method para lidar com bases de código antigas (28:31).

Processo de Feedback: Utiliza um loop de comunicação com o processo Python do Aider via comint (19:06, 22:16), permitindo que a IA execute testes e build commands automaticamente (1:31).

Observações Importantes

Prompting: O autor destaca que a qualidade do resultado depende fortemente do prompt e da quantidade de contexto injetada (7:55).

Automação: A meta é que o plugin evolua para uma interação onde a IA possa fixar bugs em loop com testes automatizados, criando um ciclo virtuoso (37:54, 39:17).

integração do Claude Code no ecossistema do Emacs, buscando uma experiência menos "wonky" (instável) que o uso em emuladores de terminal comuns. Abaixo, o resumo do workflow demonstrado:

Configuração e Ferramentas

Claude Code: A ferramenta principal usada via terminal para gerar código e gerenciar projetos.

Emacs Terminals: Comparação do uso do eat e vterm para rodar o Claude Code (1:01:01).

claude-code.el: Pacote Emacs para melhorar a integração, incluindo suporte a menus transient e execução direta dentro de buffers (1:13:35).

Monet: Uma ferramenta/pacote auxiliar que atua como um servidor para permitir que o Claude interaja com o Emacs (como um servidor MCP), possibilitando abrir arquivos e mostrar diffs (1:14:45).

Comandos e Operações Chave

Claude -R: Comando para retomar uma sessão anteriormente interrompida, mantendo o contexto (1:01:34).

Modo de Plano (Plan Mode): Ativado com Shift+Tab, utiliza o modelo Opus para planejamento e Sonnet para execução, otimizando custos e qualidade (40:36).

Autoconnect IDE: Configuração necessária para que o Claude reconheça o Emacs via Monet (1:39:39).

Atalhos no Emacs: O uso do prefixo C-c C para interagir com o Claude sem sair do editor (1:23:15).

C-c C-1, 2, 3: Atalhos para responder a prompts do Claude sem voltar ao terminal (1:30:57).

C-c C-s: Comando para enviar instruções ou continuar o fluxo de trabalho (1:27:01).

Fluxo de Trabalho (Workflow)

Iniciação: O projeto (Guile Scheme XMPP client) é iniciado no terminal e depois importado para o Emacs (1:01:25).

Integração: Instalação via use-package e configuração do claude-code.el e monet para sincronização buffer-terminal (1:14:50 - 1:15:35).

Desenvolvimento Assistido: Uso do diff tool integrado ao Emacs, que substitui a saída confusa do terminal por buffers dedicados para revisar e aceitar/rejeitar mudanças (Y ou Q) (1:40:00 - 1:41:15).

Refinamento: O Claude utiliza o contexto do projeto (via claude.md) para adaptar o comportamento e evitar comportamentos indesejados (como excesso de elogios ou emojis) (54:08, 1:33:45).

uso agêntico de LLMs no Emacs combina a flexibilidade do Lisp com a padronização do Model Context Protocol (MCP). Aqui está o destilado do processo:

Workflow de Ferramentas (Tool Use)

1. Ferramentas Nativas (Emacs Lisp)

Biblioteca Principal: gptel (pacote para interagir com LLMs).

Criação: Utiliza-se a função gptel-make-tool para definir funções Lisp como ferramentas acessíveis à IA (01:50-02:10).

Estrutura: Exige name, function, description (essencial para o LLM saber quando usar) e args (02:24-04:35).

Categorização: Agrupamento por domínios como reflection (introspecção do Emacs), file-system e web (04:35-06:15).

2. Protocolo de Contexto de Modelo (MCP)

Biblioteca: mcp.el (gerenciamento de servidores MCP) + gptel-integrations (integração direta com o gptel) (20:38, 26:30).

Servidores MCP: Permitem usar ferramentas externas (node, python, docker) via stdio (22:15-25:54).

Configuração: Adicionar servidores no mcp-servers usando comandos como npx, uvx ou docker (21:11-23:00).

Gerenciamento: M-x MCPHub para listar, iniciar e gerenciar estados dos servidores (24:56-25:20).

⚙️ Comandos e Interação

Ativação: gptel -> M-x gptel-toggle-tool-use (dash t) (09:37).

Seleção: gptel -> select-tools (a T) para escolher quais ferramentas estarão disponíveis na sessão atual (09:47).

Integração MCP: M plus dentro do menu de ferramentas do gptel para injetar servidores MCP ativos na sessão (27:15).

Confirmação: confirm-tool-calls (dash c) permite definir se a IA precisa de aprovação humana para executar ações (10:38).

💡 Melhores Práticas e Macros

Prompt de Estratégia: Utilizar um system prompt que force o modelo a um loop de Planejar -> Executar -> Refletir (12:35).

Gestão de Segredos: Uso de funções customizadas (ex: gjg-o-info-secret) integradas ao pass ou arquivos GPG para evitar hardcoding de chaves de API nos arquivos de configuração (23:27).

Performance: Cuidado com o tamanho do contexto! Enviar centenas de definições de ferramentas aumenta o consumo de tokens e a latência (29:45-30:30).

Expansão: Possibilidade de escrever seus próprios servidores MCP em Emacs Lisp usando o mcp-server-lib.el (45:15).

análise científica no Emacs, utilizando o GPTel, é um exemplo moderno de um "caderno de laboratório inteligente". Abaixo, destilo as etapas e ferramentas utilizadas:

🚀 Workflow de Análise Científica com AI Preparação e Setup

🔧 GPTel: Interface Elisp para LLMs (Claude, ChatGPT, etc.) no Emacs (0:08).

📝 Org-mode: O ambiente central de trabalho para notas e execução de código (4:12).

🐍 Jupyter/Python: Uso de blocos de código (src python) para garantir reprodutibilidade (4:21).

Exploração Inicial (Brainstorming)

🧠 Interação com LLM: O autor envia o buffer de dados para o Claude para sugerir modelos e leis de velocidade (1:34).

🔍 MCP Server (PICSSE): Ferramenta customizada pelo autor para listar funções disponíveis do pacote PICSSE, permitindo que a IA seja "consciente" das capacidades do ambiente (1:56).

Análise Quantitativa e Código

💻 Geração de Código: A IA gera código Python para realizar o fit dos dados (usando `scipy.optimize.minimize`, `solve_ivp`) (4:56).

⚡ Templates: O autor usa templates de código para garantir que o resultado seja executável em blocos de Jupyter (4:16).

Refinamento e Execução

🔧 Ajustes Manuais: O autor corrige escapes de string e formatação gerados pela IA, enfatizando a importância da revisão humana (8:33).

📊 Visualização: Uso de bibliotecas de plotagem dentro do Python para verificar a linearidade (log CA vs tempo) e a qualidade do fit (6:16).

⚙️ Comandos e Recursos Citados

🦄 Hydra: Interface de atalhos em menus para facilitar comandos repetitivos no Emacs (1:25).

⌚ Assincronismo: O GPTel roda de forma assíncrona; o autor observa a importância de indicadores visuais (o bloco fica amarelo enquanto roda) para saber quando a IA terminou o processamento (12:48).

⚠️ Notas do Autor sobre o Workflow

Alucinações: Sempre tratar o código gerado como rascunho. O autor prefere gerar código que ele possa rodar e validar (4:02).

Transparência: O autor aponta a dificuldade atual em marcar visualmente o que é código "escrito por humano" vs "escrito por IA" (10:02).

O workflow do org-ai transforma o Emacs em um assistente de IA poderoso, integrando a API da OpenAI diretamente no Org Mode. Aqui está a destilação do fluxo de trabalho:

📦 Configuração e Base

Pacote: org-ai (disponível no GitHub e MELPA).

Integração: Conecta o Emacs à API da OpenAI (GPT e DALL-E) para processamento de linguagem natural.

📝 Blocos Especiais (Org-ai blocks)

Estrutura: Inspirado nos babel source blocks, encapsula conversas dentro do buffer Org (0:33 - 1:28).

System Prompt: Permite definir o comportamento do bot logo no início do bloco.

Prefixos: O comando `me` é usado para identificar sua entrada de texto.

Comandos de Edição:

Ctrl-C + Backspace: Deleta rapidamente partes da conversa sob o cursor (1:35).

Max Token Header: Ajusta o tamanho da resposta (1:51).

🗣️ Entrada e Saída de Voz

Transcrição: Use Ctrl-C R dentro de um bloco para ditar sua entrada (2:12).

Speech Output: É possível habilitar a saída de áudio para tornar a interação uma conversa real (2:28).

Workflow de Desenvolvimento e Código

Refatoração: Selecione um bloco de código e peça ao ChatGPT para torná-lo mais eficiente; uma interface de diff será mostrada para aplicar as alterações (4:14).

Geração de Código: Solicite funções específicas ou testes unitários diretamente no buffer de código (3:24 - 4:09).

Summarization (On Region): Selecione textos em qualquer buffer (ex: artigos) e utilize o comando org-ai-on-region para resumir ou analisar o conteúdo (2:54 - 3:16).

O workflow apresentado utiliza o Emacs como um laboratório eletrônico inteligente, integrando LLMs para automação e análise de dados experimentais. Abaixo, os pilares dessa arquitetura:

Workflow de Experimentos Automatizados

Interfaces de Controle:

GPTEL: Interface Emacs Lisp que conecta o editor a modelos de linguagem (Claude, GPT, etc.) (1:06).

MCP (Model Context Protocol): Protocolo que permite ao LLM interagir com ferramentas externas e dispositivos de hardware (1:20).

Estrutura do Notebook:

Org-mode: O coração do sistema, usado para registrar intenções, design de experimentos e resultados (0:06, 0:39).

Sugar Syntax: Uso de prefixos como @claude-light ou @picsse para ativar presets específicos de sistema e ferramentas (1:44).

Automação e Execução:

Claude-Light: Instrumento remoto via MCP para controle de LEDs RGB e medições de espectro (0:15).

Tool Calling: O modelo invoca funções JSON dinamicamente para realizar medições e registrar dados diretamente nas tabelas do Org-mode (3:37).

Análise de Dados:

Python Source Blocks: O LLM gera blocos de código (Python/Jupyter) dentro do Org-mode para processar os dados obtidos (6:20).

Análise Estatística: Uso de bibliotecas como scipy para regressões lineares e análises de variância (ANOVA) (7:24, 12:50).

Comandos e Operações

Ativação: Uso de Hydras (atalhos de teclado personalizados) para disparar mensagens do GPTEL para o modelo (3:04).

Iteração: O workflow suporta design de experimentos complexos, como o Latin Hypercube Design (LHC), permitindo que o modelo planeje, execute múltiplas chamadas e interprete os resultados sequencialmente (8:24, 9:55).

Refinamento: O sistema lida com múltiplas tentativas de chamada de ferramenta (tool calls) caso a primeira tentativa falhe ou o formato dos dados precise ser ajustado (9:20, 11:34).

 Notas sobre o Futuro da Documentação

O sistema permite documentar a intenção do pesquisador, a execução automatizada pelo agente e o registro permanente da análise no arquivo de texto, tornando o laboratório reproduzível e transparente (0:44, 8:07).

O workflow de Andrew Hyatt para integrar LLMs ao Emacs foca em manter o controle dentro do editor, preservando o poder dos buffers e a filosofia do sistema. Aqui está o resumo destilado:

Bibliotecas e Ferramentas Recomendadas

GPTEL: (0:29) Recomendada por ser integrada diretamente ao Emacs, tratando o buffer como input e permitindo uma experiência fluida.

Copilot: (0:29) Destacada como uma das formas de manter o fluxo dentro do editor.

Elama: (0:34) Ferramenta para interações com LLMs mantendo a filosofia de buffers do Emacs.

Agent Shell / Claude Code: (12:17, 12:24) O Agent Shell é visto como um passo evolutivo, permitindo que o Emacs use o comment-mode para interagir com agentes, embora ainda apresente limitações em comandos complexos.

Estratégia de Workflow (O "Sweet Spot")

Foco no Buffer: A regra de ouro é utilizar ferramentas que leiam o estado atual do buffer (mesmo os não salvos), e não apenas arquivos em disco (0:27, 1:05).

Edição Híbrida: Andrew prefere fazer a edição pesada e o pensamento estruturado no Org Mode, usando-o como base para as instruções, enquanto utiliza ferramentas de terminal (como Claude Code) apenas para a execução de tarefas específicas que exigem maior contexto (13:52, 14:18).

Interação com o Shell: Em vez de depender de interfaces ricas baseadas em web (como as do VS Code), ele busca extensões que permitam prefixar comandos no eShell (ex: at) para que a IA substitua comandos complexos que ele não memoriza, mantendo a produtividade sem sair do ambiente do Emacs (18:30-19:20).

Considerações sobre a Ferramenta

Limitação de UIs: Hyatt admite que o Emacs tem limitações para exibir interfaces modernas (linhas verticais de indentação, erro visual flymake mais refinado), mas valoriza a resiliência e a capacidade de usar o editor para "tudo" (org, e-mail, shells) como um diferencial competitivo frente a editores de propósito único (9:26-10:28, 20:10-21:10).

Este workflow de "vibe coding" no Emacs foca em uma colaboração homem-máquina, utilizando LLMs para tarefas complexas enquanto mantém o controle humano. Abaixo, o destilado do processo:

Ferramentas e Infraestrutura

Editor: Emacs como o hub central.

GPTEL: Pacote principal para interação com LLMs (0:12).

MCP (Model Context Protocol): Via mcp.el para controlar terminais e arquivos (10:55).

Backend: Python com FastAPI, SQLAlchemy, Pytest (17:01).

Formato: Uso de Org Mode para documentação e planejamento (2:45).

⚙️ Workflow Passo a Passo

Definição do Sistema:

Configuração do prompt "Python Coder" (estilo especialista) para evitar elogios vazios e focar na tarefa (1:43).

Ajuste do "slider de autonomia" em 50% (5:00).

Inicialização:

Comando M-x gptel para abrir a sessão com o modelo Gemini 2.5 Pro (8:06).

Criação de arquivos e estrutura (planning.org, tasks.org) (15:36).

Comando git init (humano no comando) (15:58).

Desenvolvimento Iterativo:

LLM gera blocos de código em Org Mode (6:50).

Execução humana dos blocos (C-c C-c) para garantir ambiente correto (7:08).

Uso de ferramentas MCP para leitura/escrita de arquivos, com ressalvas sobre a fragilidade da escrita (11:08, 28:00).

Higiene de Contexto:

Limpeza manual de logs de erro e resultados de ferramentas para evitar "context clutter" (27:55).

Come-to-Jesus meeting: Reunião para resumir o estado atual e resetar o contexto quando a performance degrada (30:42).

💡 Lições Aprendidas

Contexto é tudo: O acúmulo de erros passados no histórico confunde o modelo; a limpeza frequente é obrigatória (33:52).

Human-in-the-loop: Não automatize o git commit ou refatorações radicais; valide as mudanças antes de aceitar (5:49).

Brutalidade dos File Tools: As ferramentas de edição de arquivo atuais (via MCP) são descritas como frágeis; há planos para criar uma solução customizada em Emacs Lisp (32:30).

Neste vídeo, o autor explora a integração de assistentes de IA diretamente no Emacs, transformando o editor em um ambiente mais produtivo e automatizado. Abaixo, o destilado do workflow apresentado:

🔧 Principais Ferramentas e Pacotes

gptel (1:07:55): O pacote principal utilizado para interagir via API com os modelos GPT-3.5 e GPT-4 da OpenAI diretamente de dentro dos buffers do Emacs.

copilot.el (1:07:01): Integração com o GitHub Copilot para sugestões de código em tempo real, exigindo instalação via straight.el ou clonagem direta do repositório (1:07:29).

Infraestrutura de Gerenciamento: O autor utiliza straight.el (46:34) como gerenciador de pacotes e use-package (46:32) para organizar as configurações de forma modular e limpa.

⚙️ Comandos e Macros de Fluxo

Execução da IA: O atalho C-c C-enter (9:34) é o gatilho padrão para enviar prompts para o gptel e receber respostas processadas em Org Mode.

Manipulação de Buffers: O autor utiliza macros como with-temp-buffer (1:13:43) para capturar saídas da IA ou processar arquivos em lote, embora a IA tenha demonstrado dificuldades em otimizar esse comando específico (1:17:55).

Interação com o Código: O fluxo consiste em descrever o comportamento desejado em comentários ou texto simples e permitir que a IA gere a implementação em Emacs Lisp (1:13:12).

Workflow de Desenvolvimento

Definição: O usuário descreve requisitos específicos em linguagem natural (ex: "escreva uma função que leia títulos de arquivos Org") (1:13:22).

Iteração: O usuário utiliza prompts de refinamento (ex: "isso está errado, use a função X em vez da Y") (1:29:15) para corrigir o código gerado.

Depuração: O autor utiliza o buffer de Messages (1:20:32) para verificar resultados e o Backtrace (1:15:57) para diagnosticar falhas nas funções geradas pela IA.

Configuração: Uso da IA para gerar blocos de configuração completos (ex: habilitar evil-mode, corfu, vertigo) a partir de uma lista de desejos do usuário (45:40).

Notas Importantes

Contexto: Para melhores resultados, é necessário fornecer contexto completo e ser específico com os requisitos (32:27).

Verificação: Nem sempre o código gerado é funcional (ex: a confusão entre Global Corfu Mode e Corfu Global Mode em 57:47); a revisão humana é essencial.

Privacidade e Custo: Todo o conteúdo do buffer pode ser enviado à API a cada requisição, o que pode impactar custos se o histórico for muito longo

workflow focado em produtividade no Emacs, utilizando inteligência artificial para automação e melhoria de experiência. Aqui está o resumo detalhado:

Bibliotecas e Ferramentas

GPTel: Utilizado para integração com LLMs, permitindo o uso de ferramentas para ler, resumir e traduzir documentações do Emacs via Retrieval Augmented Generation (RAG) (0:05 - 0:12).

Sinister: Novo pacote desenvolvido pelo autor para evitar o salto automático (scrolling) indesejado ao abrir mini-buffers altos, ideal para quem prefere divisões horizontais de tela (0:34 - 0:37).

De-Macs: Repositório que contém a configuração minimalista do Emacs demonstrada no vídeo (0:30 - 0:32).

Workflow e Comandos

RAG no Emacs: O autor utiliza a capacidade de introspecção do Emacs aliada ao GPTel para criar uma solução de RAG, acelerando a integração e melhoria de recursos de IA (0:06 - 0:18).

Correções de Código: Atualizações de código para a série Speedrun 01, garantindo que as dependências nativas funcionem corretamente (0:20 - 0:27).

Organização de Projetos:

O conteúdo está dividido entre o diretório speedrun (correções e rascunhos) e o diretório de-macs (configurações minimalistas e o pacote Sinister) (0:25 - 0:33).

Recursos Adicionais

Repositório do GitHub: O código e as configurações podem ser encontrados no GitHub do autor.

Sponsors: O projeto é sustentado pelo apoio da comunidade via GitHub Sponsors, servindo como acesso antecipado para o PrizeForge.

workflow para configurar IA Privada no Emacs, conforme apresentado por Aaron Grothe (0:56):

 Workflow Minimalista (MVP)
Engine Local (Backend):

 Llamafile: Baixe o executável único que contém o LLM (ex: Llama 3.2 3B). Torne-o executável (chmod +x) e rode-o (7:22).

 Server: O modelo fica rodando localmente na porta 8080 (7:55).
Interface no Emacs:

 GPTel: Instale via M-x package-install (8:07). É um buffer minimalista que não interrompe o fluxo de trabalho.

 Configuração: Adicione ao seu init.el (8:24):

Defina o backend como llama.

Configure o host como 127.0.0.1 e a port como 8080.

Comandos de Uso:

 Iniciar: M-x gptel para abrir o buffer de chat (9:31).

 Enviar: C-c C-c para enviar a consulta ao modelo (10:04).

 Workflow Avançado (Escalabilidade)

 Ollama / LM Studio: Alternativas mais robustas que o Llamafile, ideais para gerenciar múltiplos modelos e carregar pesos explicitamente na GPU (13:22).

 RAG (Retrieval Augmented Generation):

Utilize o Open Web UI para carregar documentos, códigos e whitepapers locais (13:37, 23:32).

Permite que o modelo analise seus dados privados com citações (24:36).

 Segurança: Mantenha o sistema em ambiente air-gapped ou em redes isoladas se estiver lidando com dados críticos (37:10).

 Dicas de Hardware

 GPU/NPU: Essencial para velocidade. A RAM é o gargalo principal para modelos maiores (32:21).

 Ventilação: Evite hardware que sofra thermal throttling (ex: MacBook Air) para cargas de IA prolongadas (17:15).

 Monitoramento: Use o Open Web UI para uma interface similar ao ChatGPT, porém rodando 100% local (13:47).

O workflow de "Vibe Coding" no Emacs utilizando o gptel foca na colaboração fluida com LLMs para experimentação rápida e desenvolvimento. Abaixo, os pilares do fluxo apresentado pelo criador:

 Configuração e Ferramentas

gptel: O pacote central que integra LLMs diretamente no Emacs (3:30).

Bibliotecas e Ferramentas Customizadas: Ferramentas escritas em Emacs Lisp que permitem ao LLM interagir com o ambiente (14:34), como:

Web Tools: Busca e leitura de páginas da web (15:23).

Reflection Tools: Acesso a pacotes, funções, man pages e documentação do sistema (15:35).

Home/Util Tools: Integrações locais como clima ou automação residencial (15:51).

⚙️ Comandos e Funcionalidades Chave

Reasoning (Raciocínio): O modelo exibe seu processo de pensamento entre tags org-mode (ex: `#+begin_reasoning`), permitindo ao usuário acompanhar a lógica antes da resposta (4:31). O modo ignore é recomendado para evitar que o modelo se confunda com seu próprio histórico de pensamento (6:00).

Tool Usage (Uso de Ferramentas): Configurado via Control-C Return (14:19). Pode ser definido como confirm (exige aprovação humana) ou no-confirm (autônomo) (16:15).

Contexto de Arquivos: Capacidade de anexar arquivos específicos ao buffer de chat (Add files to context) para que o modelo entenda a base de código do projeto (21:41).

Navegação e Foco: Uso de org-mode com pacotes como org-powerslides para navegar por headlines, isolar blocos de código e manter o foco durante a codificação (27:27).

🚀 O Workflow de Vibe Coding

Setup do Chat: Iniciar um buffer do gptel e definir o sistema (System Prompt) como um "parceiro erudito" em vez de um assistente subserviente (7:22).

Iteração: Apresentar um problema ou ideia, permitindo que a IA sugira refatorações ou descubra bugs (28:23).

Refinamento: Avaliar as sugestões (como eliminar dependências externas ou alterar a estrutura de funções) e implementar as mudanças diretamente no Emacs (29:17).

Experimentação: Transformar conceitos em novas funcionalidades (ex: converter um pacote em um minor mode) com a ajuda da IA para gerar o boilerplate e lógica inicial (35:10).

Este workflow no Emacs para integração com LLMs transforma seu editor em um ambiente de AI Pair Programming. Abaixo, o destilado das ferramentas e práticas abordadas no vídeo:

🛡️ Segurança e Configuração

Auth Info & GPG: Nunca deixe chaves de API expostas no init.el. Utilize o arquivo `~/.authinfo.gpg` para armazenar credenciais de forma criptografada (10:14-11:33).

Gerenciamento: Recomenda-se o uso de straight.el para pacotes e o sistema nativo use-package para organização (4:50-6:22).

🤖 Principais Ferramentas (Pacotes)

GitHub Copilot:

copilot.el: Focado em autocompletar código inline (18:57, 1:12:14).

copilot-chat.el: Interface de chat para interagir, otimizar e documentar código (22:26, 34:02).

GPTel:

Permite interagir com múltiplos modelos (Google Gemini, OpenAI, locais). Funciona tanto em buffers dedicados quanto via gptel-send em qualquer lugar do editor (38:31, 43:31).

Adomax (ou Aider):

O mais poderoso: integração total com Git. Ele gera mudanças, aplica diffs, cria commits e gerencia o contexto do projeto (52:42, 1:02:26).

⚙️ Comandos e Fluxo de Trabalho

Diagnóstico: Use copilot-diagnose para verificar se a autenticação e o servidor estão operantes (20:24, 1:11:18).

Arquitetura de Edição: O Architect Mode no Adomax utiliza um processo de dois passos: uma LLM de raciocínio processa o prompt e a outra de edição traduz o plano em código real (57:26).

Contexto: Todos os pacotes suportam o envio de buffers ou regiões de código como contexto. O Adomax, especificamente, detecta automaticamente a raiz do projeto Git (1:01:43).

💡 Dicas de Ouro

Privacidade: Tenha cautela com dados sensíveis; interações em planos gratuitos ou modelos padrão geralmente são usadas para treinamento das empresas (6:31-7:58).

Personalização: O apresentador utiliza um remapeamento radical do teclado (usando ; como prefixo), sugerindo que cada usuário deve adaptar o workflow ao seu próprio init.el em vez de copiar snippets cegamente (4:37-5:47).

O workflow apresentado pelo System Crafters foca em um sistema de automação pessoal baseado em agentes autônomos (LLMs), utilizando Claude Code e o ecossistema Sigil. Aqui está o destilado desse sistema:

🔧 Arquitetura do Sistema (Ops)

Claude Ops (claude-ops.sh): O ponto de entrada que inicializa sessões do Claude Code com configurações específicas e injeta prompts de contexto (19:16 - 20:31).

MCP (Model Context Protocol): A espinha dorsal para conectar ferramentas externas ao LLM, permitindo controle de e-mail, agenda, tarefas e conhecimento (16:33).

Folio: Servidor MCP de gestão de conhecimento que utiliza arquivos Markdown locais como base de dados estruturada, permitindo busca por tags, datas e cross-linking (22:43 - 24:44).

🤖 Automação e Agentes

Líder vs. Trabalhadores: O sistema utiliza um agente "Líder" para gerenciar o fluxo e instanciar "Trabalhadores" (sub-agentes) que executam tarefas isoladas em novas sessões de T-mux (22:04, 1:13:06, 1:15:49).

Courier: Ferramenta MCP para comunicação/coordenação entre o agente líder e os trabalhadores (20:45).

Minder: Servidor MCP responsável por agendamento de tarefas, rodando comandos baseados em tempo de forma assíncrona (42:17).

📝 Gestão de Conhecimento e Tarefas

Notas Atômicas: O sistema exige que cada sessão de trabalho crie/atualize notas no formato Folio, garantindo que a base de conhecimento cresça continuamente (130:30 - 131:08).

Task Briefing: Antes de iniciar, o sistema gera um arquivo de briefing contendo critérios de aceitação e diretrizes de design, que o trabalhador lê antes de começar (113:46 - 114:09).

🔧 Ferramentas e Bibliotecas

Sigil: Linguagem e ambiente baseados em Scheme (com extensões próprias como dicts) usada para desenvolver o próprio sistema (16:55, 126:34).

Easel: Wrapper das APIs Cairo usado para automação de design, como geração de miniaturas para o canal (25:01).

Bureau: Servidor MCP dedicado para triage de e-mail e gestão de calendário (27:36).

Harold (em construção): Novo servidor MCP para processamento de RSS Feeds (104:32, 138:39).

⚡ Comandos e Comportamento

Simplificação: Pré-commit obrigatório que formata e simplifica o código antes de subir para o repositório (130:27).

GitOps: Todo o estado do sistema (notas, tarefas, código) é versionado via Git, garantindo redundância e histórico de decisões (16:50, 23:51).

Cache: Uso de SQLite para indexar as notas do Folio, otimizando a velocidade de busca para o agente (131:14).
